

Applied Deep Learning

Chapter 6: Recurrent NNs

Ali Bereyhi

`ali.bereyhi@utoronto.ca`

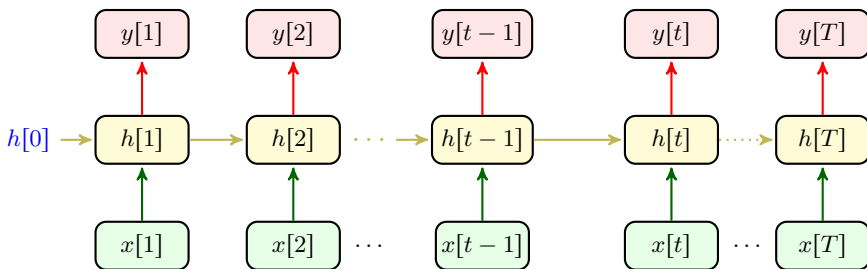
Department of Electrical and Computer Engineering
University of Toronto

Winter 2026

Principle of Gating

To understand the idea of *Gating*, let's get back to our *basic RNN*

- 1 We start with *hidden state* $h[0]$
- 2 We have $h[t] = f(w_1x[t] + w_mh[t-1])$
- 3 We have $y[t] = f(w_2h[t])$



Looking at $h[t]$ as *memory*, we can say *we are always updating the memory*

Principle of Gating

Recall our motivating example: we wanted to predict the *next word*

$x[t-6]$ $x[t-5]$ $x[t-4]$ $x[t-3]$ $x[t-2]$ $x[t-1]$ $x[t]$
... Julia has been nominated to receive Alexander von Humboldt Prize for her

Should we update the memory *all the way* from Julia?

- Obviously No!
 - ↳ We should stop updating at Julia, since it is something we should remember
- How can we do it? *Let's try a thought experiment*

Principle of Gating

Say we access to a sequence $u[t] \in [0, 1]$: assume the following happens

- ↳ We arrive at “*Julia*” at time t_0 , i.e., $x[t_0] \propto$ “*Julia*”
- ↳ At time t_0 , we have $u[t_0] = 1$
- ↳ We want to predict at $t + 1$
- ↳ From $t_0 + 1$ to t , we have $u[t_0 + 1] = \dots = u[t] = 0$

Now, we update our memory like this

- 1 We start with hidden state $h[0]$
- 2 We compute $\tilde{h}[t] = f(w_1 x[t] + w_m h[t - 1])$
- 3 We update $h[t] = u[t] \tilde{h}[t] + (1 - u[t]) h[t - 1]$
- 4 At each time, we have $y[t] = f(w_2 h[t])$

Let's see what happens to the *memory*

Principle of Gating

At time t_0 , we can say

- We have $x[t_0] \propto$ "Julia" and compute $\tilde{h}[t_0] = f(w_1 x[t_0] + w_m h[t_0 - 1])$
 - ↳ $\tilde{h}[t_0]$ has fresh memory about "Julia"
 - ↳ Since $u[t_0] = 1$, we update as $h[t_0] = 1 \times \tilde{h}[t_0] + 0 \times h[t_0 - 1] = \tilde{h}[t_0]$
 - ↳ RNN has a fresh memory about "Julia"

At the next time, i.e., $t_0 + 1$, we have

- No matter what $\tilde{h}[t_0 + 1]$ is we have $u[t_0 + 1] = 0$
 - ↳ We update as $h[t_0 + 1] = 0 \times \tilde{h}[t_0 + 1] + 1 \times h[t_0] = h[t_0] = \tilde{h}[t_0]$
 - ↳ We still have a fresh memory about "Julia"

This repeats from $t_0 + 1$ till t , so at time t

- No matter what $\tilde{h}[t]$, we have $u[t] = 0$
 - ↳ We update as $h[t] = 0 \times \tilde{h}[t] + 1 \times h[t - 1] = h[t - 1] = \dots = \tilde{h}[t_0]$
 - ↳ We still have a fresh memory about "Julia"

Principle of Gating: *Updating via Gates*

$u[t]$ *gates the memory*: it decides how much *memory* we should *pass* and *forget*

- We update $h[t] = u[t]\tilde{h}[t] + (1 - u[t])h[t - 1]$
- + How does it help with vanishing gradient?
- Well! It is *implicitly* making *skip connections through time*

Recall that with standard BPTT, we have for $i = t, t - 1, \dots, t_0$

$$\frac{\partial h[i]}{\partial w_m} = f'(z[i]) \left(h[i - 1] + w_m \frac{\partial h[i - 1]}{\partial w_m} \right)$$

But, now we skip multiple time slots, as we have for $i = t, t - 1, \dots, t_0$

$$\frac{\partial h[i]}{\partial w_m} = \frac{\partial h[i - 1]}{\partial w_m}$$

Principle of Gating: A Generic Gate

- + Sounds inspiring! But, how could you get $u[t]$?
- Well! Like what we did the whole time: we *learn* it!

Gate

Let $\mathbf{x}[t]$ be input and $\mathbf{h}[t - 1]$ be last hidden state: a gate $\Gamma[t]$ is computed as

$$\Gamma[t] = \sigma(\mathbf{W}_{\Gamma,\text{in}}\mathbf{x}[t] + \mathbf{W}_{\Gamma,\text{m}}\mathbf{h}[t - 1] + \mathbf{b}_{\Gamma})$$

where $\sigma(\cdot)$ is sigmoid function, and $\mathbf{W}_{\Gamma,\text{in}}$, $\mathbf{W}_{\Gamma,\text{m}}$ and $\mathbf{b}_{\Gamma}$ are *learnable*¹

- + Why do we use sigmoid function?
 - Simply because *it is between 0 and 1*
- + What should we set the dimension of $\Gamma[t]$?
 - Same as the variable (*memory* component) that *we want to gate*

¹We are going to drop *bias* and you all know why!

Practical Gated Architectures

There are various *gated* architectures: we look into two of them

- 1 Gated Recurrent Unit (GRU)
- 2 Long Short-Term Memory (LSTM)

Before we start, let's recall their basic RNN counterpart

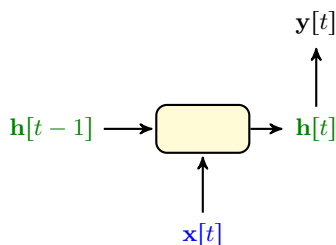
Basic RNN Counterpart

Say we set activation to $f(\cdot) \rightsquigarrow$ we usually set it to $\tanh(\cdot)$

- 1 Start with an initial *hidden state*
 $\hookrightarrow$ we *can learn* $\mathbf{h}[0]$
- 2 Compute memory as $\mathbf{h}[t] = f(\mathbf{W}_1 \mathbf{x}[t] + \mathbf{W}_m \mathbf{h}[t-1])$
 $\hookrightarrow$ we *can learn* $\mathbf{W}_1$ and $\mathbf{W}_m$
- 3 Compute output $\mathbf{y}[t] = f_{\text{out}}(\mathbf{W}_2 \mathbf{h}[t]) \rightsquigarrow f_{\text{out}}$ and f could be *different*
 $\hookrightarrow$ we *can learn* $\mathbf{W}_2$

Classical Diagram: *Hidden Layer as Unit*

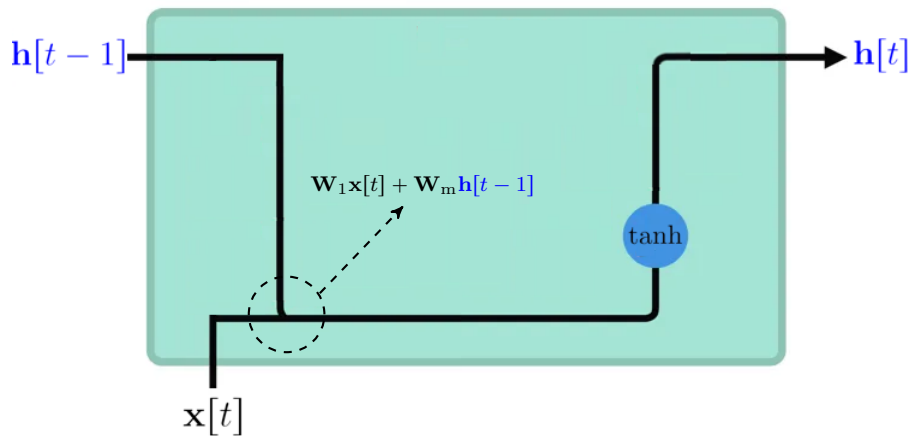
When we study a **gated architecture**: it is **common** to look at the **hidden layer as a unit** which takes **some inputs** and returns **some outputs**



We are mainly interested on **this block**: we want to know that given **last state** and **new input**

- 1 How does **this unit** **update hidden state**?
- 2 What components are passed to the **next time interval**?
 - ↳ Here, we have only **$h[t]$**
 - ↳ But, we may have **other components**
 - ↳ We will see it in **LSTM**

Classical Diagram: *Basic RNN*



Practical Gated Architectures: *Gated Recurrent Unit*

Gated Recurrent Unit (GRU)

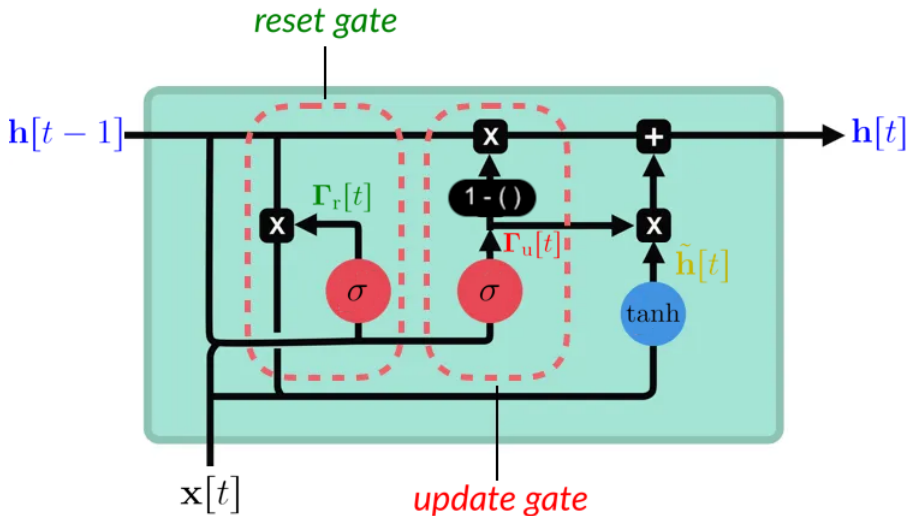
Say we set activation to $f(\cdot) \rightsquigarrow$ we usually set it to $\tanh(\cdot)$

- ① Start with an initial *hidden state*
- ② Compute *update gate* $\Gamma_u[t] = \sigma(\mathbf{W}_{u,\text{in}}\mathbf{x}[t] + \mathbf{W}_{u,\text{m}}\mathbf{h}[t-1])$
- ③ Compute *reset gate* $\Gamma_r[t] = \sigma(\mathbf{W}_{r,\text{in}}\mathbf{x}[t] + \mathbf{W}_{r,\text{m}}\mathbf{h}[t-1])$
- ④ Compute *actual memory* $\tilde{\mathbf{h}}[t] = f(\mathbf{W}_1\mathbf{x}[t] + \mathbf{W}_m\Gamma_r[t] \odot \mathbf{h}[t-1])$
- ⑤ Update *hidden state* as $\mathbf{h}[t] = (1 - \Gamma_u[t]) \odot \mathbf{h}[t-1] + \Gamma_u[t] \odot \tilde{\mathbf{h}}[t]$
- ⑥ Compute output $\mathbf{y}[t] = f_{\text{out}}(\mathbf{W}_2\mathbf{h}[t]) \rightsquigarrow f_{\text{out}}$ and f could be *different*

Or we could give $\mathbf{h}[t]$ to a *new layer*: for instance a *new GRU* whose *input is* $\mathbf{h}[t]$ and has its own *state*

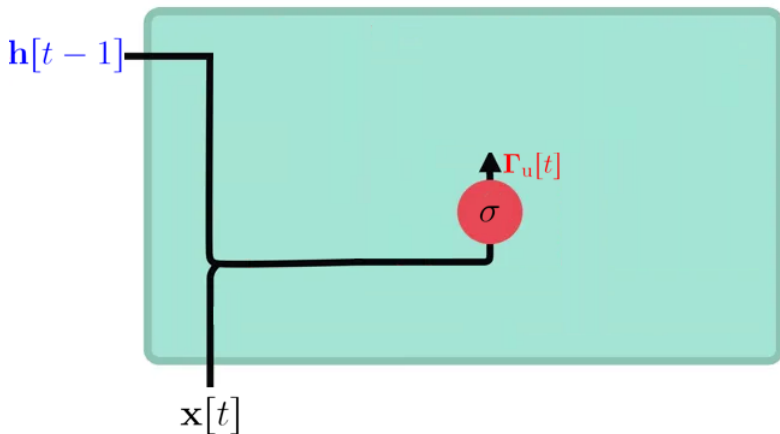
Practical Gated Architectures: GRU

This is what's going on in a GRU cell



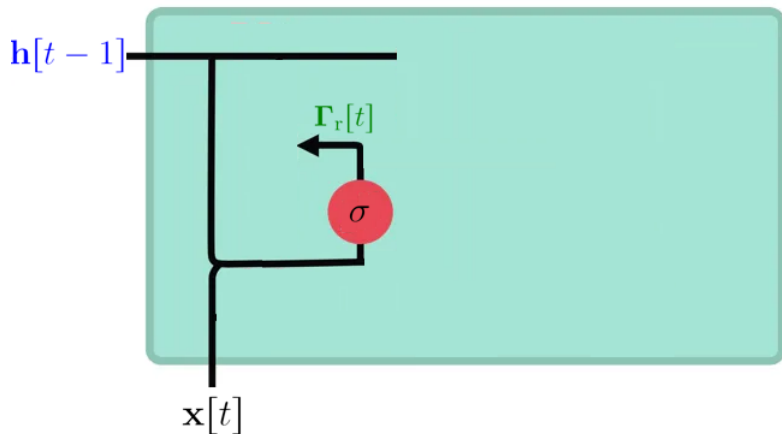
Practical Gated Architectures: GRU

Compute *update gate* $\Gamma_u[t] = \sigma(\mathbf{W}_{u,\text{in}}\mathbf{x}[t] + \mathbf{W}_{u,\text{m}}\mathbf{h}[t-1])$



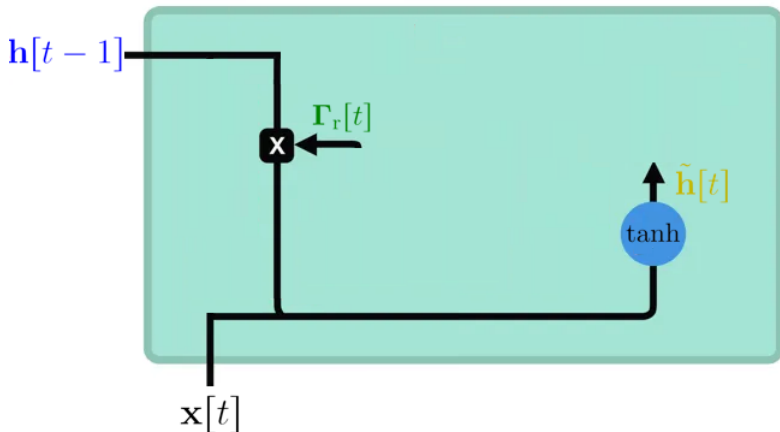
Practical Gated Architectures: GRU

Compute *reset gate* $\Gamma_r[t] = \sigma(\mathbf{W}_{r,\text{in}}\mathbf{x}[t] + \mathbf{W}_{r,\text{m}}\mathbf{h}[t-1])$



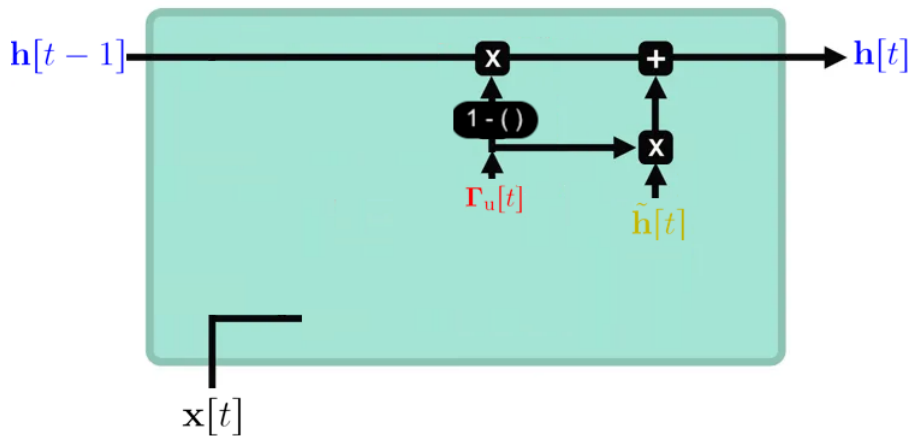
Practical Gated Architectures: GRU

Compute *actual memory* $\tilde{\mathbf{h}}[t] = f(\mathbf{W}_1 \mathbf{x}[t] + \mathbf{W}_m \mathbf{\Gamma}_r[t] \odot \mathbf{h}[t - 1])$



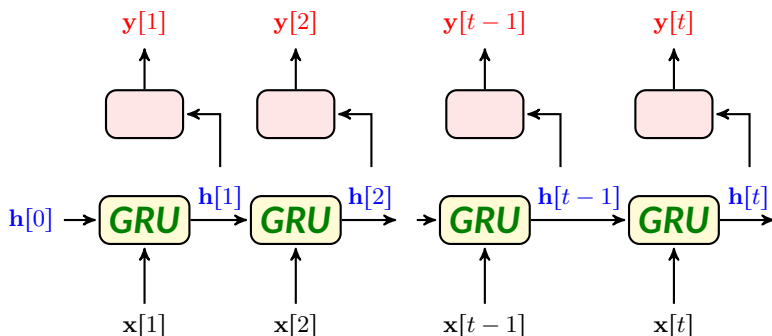
Practical Gated Architectures: GRU

Update *hidden state* as $\mathbf{h}[t] = (1 - \mathbf{\Gamma}_u[t]) \odot \mathbf{h}[t-1] + \mathbf{\Gamma}_u[t] \odot \tilde{\mathbf{h}}[t]$



GRU: Forward Pass

Starting from an *initial state*: GRU applies the first 5 steps *each time*



GRU: Backward Pass

Say we finished *forward pass* at time t . We now want to find $\nabla_{\mathbf{W}} \hat{R}$ for some $\mathbf{W}$ that is *inside GRU*, e.g., $\mathbf{W}_{u,m}$: we start *backpropagating* from $\nabla_{\mathbf{y}[t]} \hat{R}$

$$\nabla_{\mathbf{W}} \hat{R} = \nabla_{\mathbf{y}[t]} \hat{R} \circ \nabla_{\mathbf{W}} \mathbf{y}[t]$$

① We know $\mathbf{y}[t] = f_{\text{out}}(\mathbf{W}_2 \mathbf{h}[t])$

$$\nabla_{\mathbf{W}} \mathbf{y}[t] = \nabla_{\mathbf{h}[t]} \mathbf{y}[t] \circ \nabla_{\mathbf{W}} \mathbf{h}[t]$$

② We know that $\mathbf{h}[t] = (1 - \mathbf{r}_u[t]) \odot \mathbf{h}[t-1] + \mathbf{r}_u[t] \odot \tilde{\mathbf{h}}[t]$

$$\begin{aligned} \nabla_{\mathbf{W}} \mathbf{h}[t] = & \nabla_{\mathbf{r}_u[t]} \mathbf{h}[t] \circ \nabla_{\mathbf{W}} \mathbf{r}_u[t] + \nabla_{\mathbf{h}[t-1]} \mathbf{h}[t] \circ \nabla_{\mathbf{W}} \mathbf{h}[t-1] \\ & + \nabla_{\tilde{\mathbf{h}}[t]} \mathbf{h}[t] \circ \nabla_{\mathbf{W}} \tilde{\mathbf{h}}[t] \end{aligned}$$

③ ...

Practical Gated Architectures: Long Short-Term Memory

Long Short-Term Memory (LSTM)

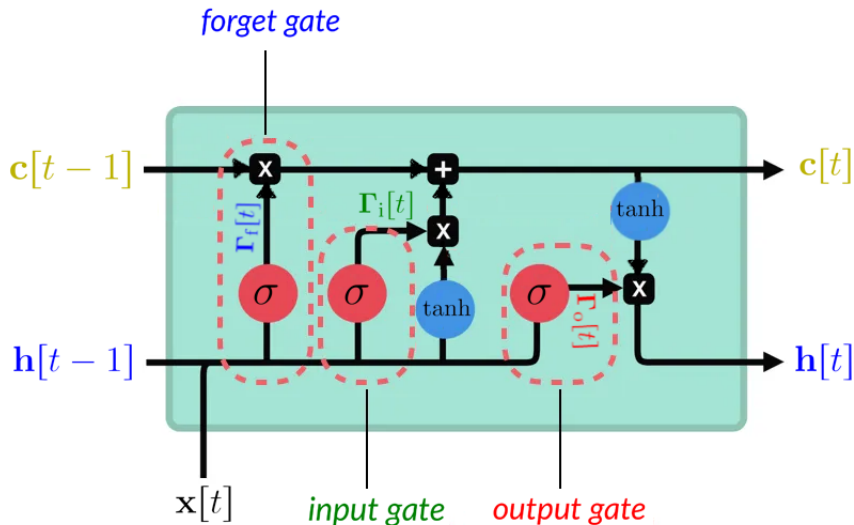
Say we set activation to $f(\cdot) \rightsquigarrow$ we usually set it to $\tanh(\cdot)$

- ① Start with initial *hidden state* and *cell state*
- ② Compute *forget gate* $\Gamma_f[t] = \sigma(\mathbf{W}_{f,\text{in}}\mathbf{x}[t] + \mathbf{W}_{f,\text{m}}\mathbf{h}[t-1])$
- ③ Compute *input gate* $\Gamma_i[t] = \sigma(\mathbf{W}_{i,\text{in}}\mathbf{x}[t] + \mathbf{W}_{i,\text{m}}\mathbf{h}[t-1])$
- ④ Compute *output gate* $\Gamma_o[t] = \sigma(\mathbf{W}_{o,\text{in}}\mathbf{x}[t] + \mathbf{W}_{o,\text{m}}\mathbf{h}[t-1])$
- ⑤ Compute *actual cell state* $\tilde{\mathbf{c}}[t] = f(\mathbf{W}_1\mathbf{x}[t] + \mathbf{W}_m\mathbf{h}[t-1])$
- ⑥ Update *cell state* as $\mathbf{c}[t] = \Gamma_f[t]\mathbf{c}[t-1] + \Gamma_i[t] \odot \tilde{\mathbf{c}}[t]$
- ⑦ Update *hidden state* as $\mathbf{h}[t] = \Gamma_o[t] \odot f(\mathbf{c}[t])$
- ⑧ Compute output $\mathbf{y}[t] = f_{\text{out}}(\mathbf{W}_2\mathbf{h}[t]) \rightsquigarrow f_{\text{out}}$ and f could be *different*

Or we could give $\mathbf{h}[t]$ to a *new layer*: for instance a *new LSTM* whose *input* is $\mathbf{h}[t]$ and has its own *hidden and cell states*

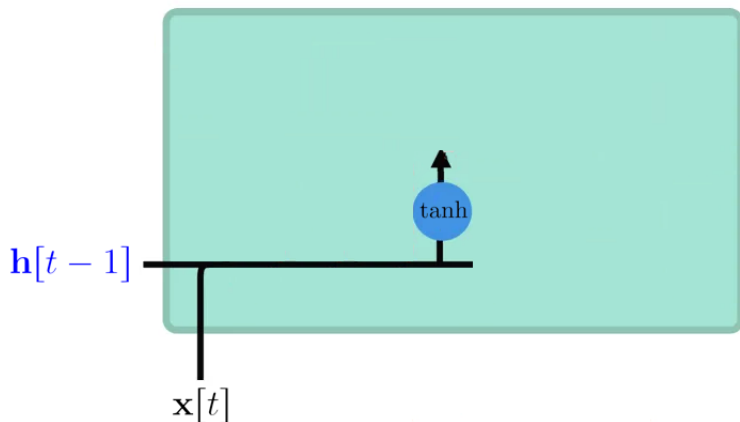
Practical Gated Architectures: *LSTM*

This is how inside an *LSTM unit* looks like



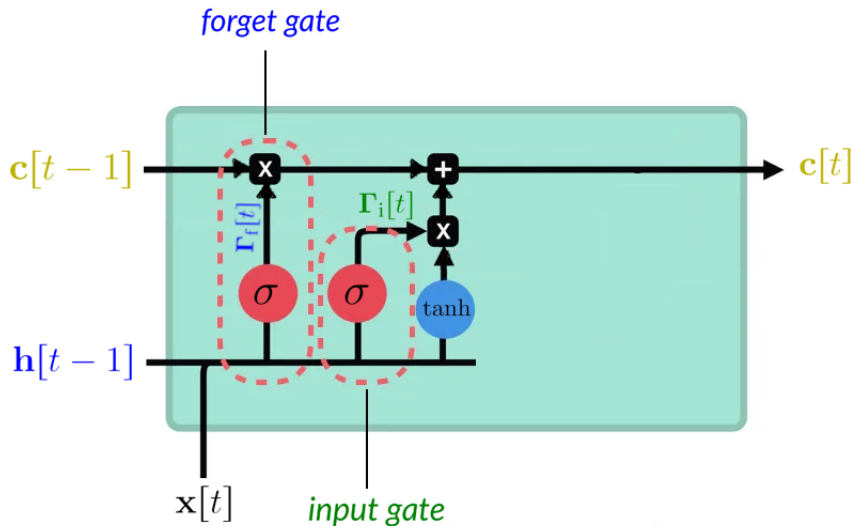
Practical Gated Architectures: *LSTM*

Actual cell state $\tilde{\mathbf{c}}[t] = f(\mathbf{W}_1 \mathbf{x}[t] + \mathbf{W}_m \mathbf{h}[t-1])$



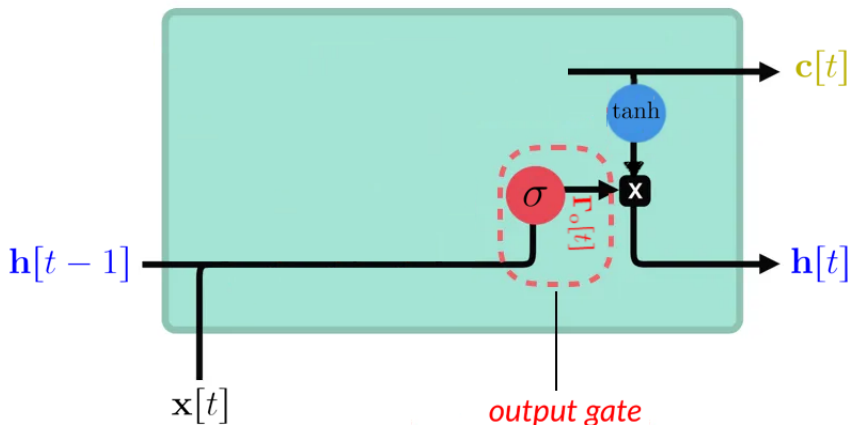
Practical Gated Architectures: *LSTM*

We use *forget gate* and *update gate* to update *cell state*



Practical Gated Architectures: LSTM

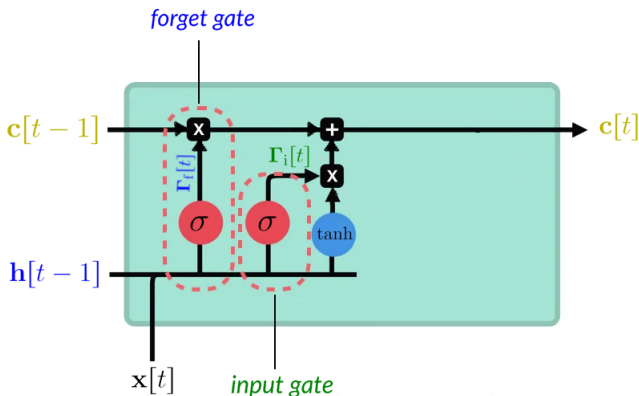
We use *output gate* to control fellow of memory to the *hidden state*



Practical Gated Architectures: LSTM

Intuitively, the gates in **LSTM** impact the *flow of information* as follows

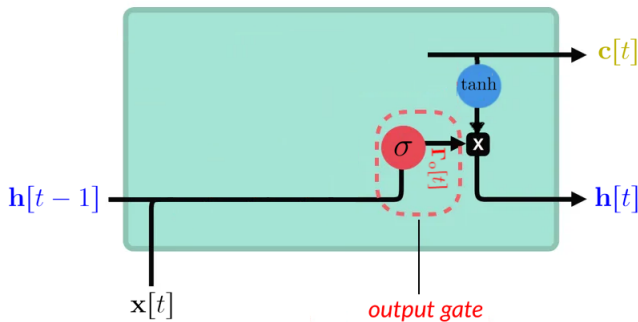
- **Forget gate** controls how much we forget from *last state*
 - ↳ Assume $\Gamma_f[t] = 0$: then, we remember nothing of $c[t-1]$
- **Input gate** controls how much we remember from *new cell state*
 - ↳ Assume $\Gamma_i[t] = 0$: then, we remember nothing of $\tilde{c}[t]$



Practical Gated Architectures: LSTM

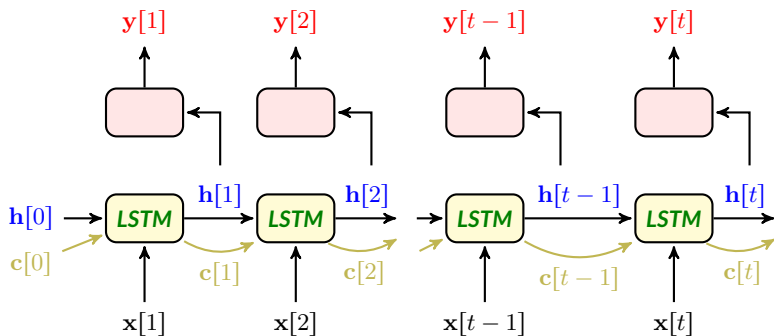
Intuitively, the gates in LSTM impact the *flow of information* as follows

- **Output gate** controls how much we let from *updated state* to *go out*
 - ↳ Assume $\Gamma_0[t] = \mathbf{0}$: then, we send nothing of $\mathbf{c}[t]$ out



LSTM: Forward Pass

Starting from *initial hidden and cell state*: LSTM passes forward as



Pay Attention

Note that unlike other architectures, LSTM does **not** keep all memory inside *hidden state* but it carries it also in *cell state*. This state is only for *memory* and is not directly used by higher layers, e.g., *output layer of the NN*

LSTM: Backward Pass

Say we finished *forward pass* at time t . We now want to find $\nabla_{\mathbf{W}} \hat{R}$ for some $\mathbf{W}$ that is *inside LSTM*, e.g., $\mathbf{W}_{i,m}$: we start *backpropagating* from $\nabla_{\mathbf{y}[t]} \hat{R}$

$$\nabla_{\mathbf{W}} \hat{R} = \nabla_{\mathbf{y}[t]} \hat{R} \circ \nabla_{\mathbf{W}} \mathbf{y}[t]$$

① We know $\mathbf{y}[t] = f_{\text{out}}(\mathbf{W}_2 \mathbf{h}[t])$

$$\nabla_{\mathbf{W}} \mathbf{y}[t] = \nabla_{\mathbf{h}[t]} \mathbf{y}[t] \circ \nabla_{\mathbf{W}} \mathbf{h}[t]$$

② ...

Suggestion

Try writing it once to see the impact of gates!